CLICK-A-THON 2026 · INMOBI TRACK

From alert to answer

A metric moves. In under a minute the system names the segment responsible, proves it, states what it ruled out — and shows its working.

ClickHouse Cloud · ClickStack · Langfuse · LibreChat

THE PROBLEM

An alert says *that*. A human spends hours on *why*.

Revenue across thousands of app, device, geo and advertiser combinations. The data already holds the answer — the bottleneck is a person slicing dashboards one dimension at a time.

9M
ad events, 5 weeks

12
dimensions to search

~2,500
candidate segments

THE TRAP THAT DEFINES THE DESIGN

On Jun 21 every segment fell 45% together

SEGMENT	CHANGE	RANKED BY ABSOLUTE DELTA
ad_format = banner	−44%	#1 — largest segment
region = NAM	−44%	#2 — second largest
category = ecommerce	−44%	#3

Rank by delta and you name **banner** with total confidence. It is a fabrication — the movement was global. The rubric punishes that harder than a miss.

> *A single fabricated figure costs more than a missed anomaly.*

SHAPE OF THE SYSTEM

ClickHouse computes. The model only writes the sentence.

```
ad_events_raw  9,000,000 rows, source-faithful
      ↓  dimensions folded in at load — the drill-down never joins
ad_events      denormalized, LowCardinality
      ↓  MATERIALIZED VIEW
segment_cube   2.9M rows · 17 MiB · 12 dimensions
      ↓
1 DETECT → 2 SCAN → 3 REFINE → 3b SECOND LEVEL → 4 EVIDENCE
                                                      ↓
                              5 NARRATE → guard → 6 ACT
```

Every stage is SQL. The LLM appears once, at the end, and receives finished strings.

THE CUBE

One query scans all twelve dimensions

A materialized view ARRAY JOINs each event into one row per (dimension, value), plus a __total__ row so a segment and its baseline arrive in the same scan.

```
('__total__','all'), ('ad_format', ad_format), ('category', category),
('os_version', os_version), ('country', country), … one line per dimension
```

69 ms
12-dimension sweep

17 MiB
smaller than the parquet it summarises

74×
fewer rows read after ordering by cardinality

DETECTION

Four questions, stated so anyone can check them

WHAT SHOULD IT HAVE BEEN?

The middle value of the same hour, same weekday, three weeks back. Weekends cancel by construction.

HOW MUCH DOES IT NORMALLY WOBBLE?

Median absolute deviation — an anomaly can't inflate the very yardstick used to detect it.

IS THIS FAR OFF?

Effect ÷ wobble. Past four, report it. Run at hourly *and* daily grain.

WHO CAUSED IT?

Not here — that is the scan's job, and keeping them apart is what makes silence possible.

Daily grain is not optional: the eCPM incident is **−2.4% across four days** — invisible per hour, obvious per day.

ATTRIBUTION

Three questions. Only the third can name a cause.

IS IT REAL?

ClickHouse's native proportionsZTest. Android 15 scores **z = −187** against a next-best of −52.

IS IT DISTINCTIVE?

The segment's change measured against the *population's* change — not its absolute delta.

IS IT RESPONSIBLE?

Remove each candidate and remeasure. Whichever leaves least behind is the cause.

A segment can beat the population handsomely and still drive nothing. One app scored **−33%** on eCPM and explained **24%** of the movement. Deviation says *unusual*; only the residual says *responsible*.

TRUSTWORTHINESS

The model cannot invent a number, because it has none

It receives a JSON of **finished strings** — no database handle, no raw rows, no arithmetic. A figure not in that JSON has nowhere to come from.

```
"change": "−4.4%",        not  −0.04382
"culprit": { "dimension": "device OS version",   not  os_version
             "its_change": "−44.8%" }
```

Then every numeric token in the prose is checked back against the evidence. Two failures and a deterministic template ships instead.

Not discouraged — structurally impossible, and verified. Tested against a re-rounded 47.7% and a computed 106,359; both caught.

And every trace carries **the SQL actually run** — its parameters, rows returned, elapsed ms. A judge checks the working, not a claim about it.

RESULTS · ONE DASHBOARD, ONE ROW PER ANOMALY

What broke, and why, in one place

METRIC	WINDOW	MOVED	VERDICT	CAUSE	EXPLAINS
request volume	Jun 21	−45.8%	global	no segment responsible	0%
fill rate	Jun 23–25	−4.3%	localized	os_version = Android 15	98.2%
revenue	Jun 21	−46.4%	global	no segment responsible	3.2%
eCPM	Jun 19–22	−2.4%	localized	category = finance	97.9%
fill rate	Jun 29–30	−1.1%	partial	country = JP → iOS 18.1	45.9%

explains — how much of the movement disappears when that segment is removed. 98% means it *is* the cause. 0% means nothing is.

Four planted movements, four found, zero false positives, no weekend flagged. Full report in **20 seconds**.

THE RESULT WE ARE PROUDEST OF

Knowing when to say nothing

> *Global — every segment moved with the population. No single segment is responsible.*

On Jun 21 the worst offender sits **2.9%** from the population once normalised. Nothing stands clear, so nothing is named.

Declining to localise is the hardest result to produce and the easiest to get wrong. Most systems will name the biggest segment. Ours reports two global movements and refuses both.

WHEN ONE DIMENSION ISN'T THE ANSWER

The cause is often an intersection

REPORTED	FILL RATE	EXPLAINS
country = JP	0.784 → 0.776	47%
JP × iOS 18.1	0.787 → **0.394**	essentially all · z = −65
JP × every other OS	flat	—
iOS 18.1 outside JP	−6.6%	mild

The system already knew it hadn't finished — that is what partial means. It now drills inside the accused segment, **but only when the residual says to**. Two-dimension search is far more surface for a spurious result.

CLOSING THE LOOP

The alert no longer needs to know the answer

```
detect.sql on a schedule  →  detections table
                                    ↓
HyperDX: "did the detector find anything?"  threshold 0
                                    ↓
webhook → GitHub → self-hosted runner → full investigation
                                    ↓
                    6 ACT — pin the culprit in HyperDX
```

The old alert counted *Android 15 non-fills above 150* — it only worked because we already knew the answer.

Proven segment-agnostic: we broke **country = JP**, which appears nowhere in any configuration. Caught at –19.4 wobbles and correctly attributed.

And it ends somewhere a human looks — a HyperDX dashboard listing every anomaly with its cause, plus a saved view pinned per finding. **Nobody copied anything across.**

WOULD THIS SURVIVE CONTACT WITH PRODUCTION?

Built to be wrong loudly, not quietly

12 REGRESSION TESTS

Recall, precision, the crying-wolf case, the numeric guard. **Proven to fail** — run against a holdout they produce nine readable failures, not a vacuous pass.

THE UNSEEN INCIDENT, FOR REAL

Sealed data landed, zero recalibration. Its regenerated dimensions agree with the originals at **chance** (0.126 vs 0.125) — so each era keeps its own attribution, and all three known incidents reproduce unchanged. Found **iOS 17.5**, **iPhone 14**, and one it refused to localize.

REPLAY

Steps a cutoff through the timeline so the detector sees only the past. **6-hour time-to-detect.**

INSPECTABLE

Every trace carries the SQL, its parameters and what it returned — so a conclusion can be re-run, not just read.

DEGRADES, DOESN'T CRASH

A baseline ladder for short slices; tracing and acting can fail without taking the diagnosis with them.

THE STACK, AND WHAT IT HOLDS

One ClickHouse does the work. The rest observe.

CLICKHOUSE CLOUD

Every figure in every diagnosis is computed here. MVs, AggregatingMergeTree, LowCardinality, window functions, proportionsZTest.

CLICKSTACK / HYPERDX

Reads Cloud directly. Sources, saved searches, and the alert that starts an investigation.

LANGFUSE

One trace per run, investigations nested inside. Open it and follow what was checked, in what order.

LIBRECHAT

An agent over the ClickStack MCP, told the same attribution rule the engine enforces.

`stack/docker-compose.yml` stands the whole plane up from upstream images — nine images, none built by us.

CLOSE

Every number is checked. Every claim is traced. Silence is a valid answer.

THE LIST

HyperDX → *RCA — what broke and why*

THE REASONING

Langfuse → one trace per run, with the SQL

THE CODE

github.com/ss-loves-AC/parts-watcher-2